Dataset used :

[1]:

	Order_ID	Customer_Name	Product	Quantity	Unit_Price	Order_Date	Region
0	1001	rahul sharma	laptop	2	55000	2026-01-05	hyderabad
1	1002	PRIYA REDDY	MOBILE	3	18000	06/01/2026	KHAMMAM
2	1003	Amit Kumar	headphones	5	1500	2026-01-07	warangal
3	1004	sneha rao	TABLET	1	22000	08/01/2026	HYDERABAD
4	1005	NaN	Laptop	4	55000	2026-01-09	khammam
5	1006	KIRAN	mobile	five	18000	10/01/2026	Warangal
6	1007	ananya reddy	Headphones	2	1500	2026-01-11	hyderabad
7	1008	RAHUL SHARMA	laptop	3	unknown	12/01/2026	KHAMMAM
8	1009	arjun reddy	TABLET	1	22000	31/02/2026	warangal
9	1010	meena devi	laptop	2	55000	2026-01-14	HYDERABAD
10	1011		NaN	6	18000	15/01/2026	NaN
11	1012	Vikram Rao	MOBILE	4	NaN	2026-01-16	khammam
12	1013	sai krishna	headphones	NaN	1500	17/01/2026	Warangal
13	1014	Priya Reddy	tablet	2	22000	2026-01-18	hyderabad
14	1015	naveen kumar	Laptop	3	55000	19/01/2026	KHAMMAM
15	1016	divya sharma	mobile	5	1558000	2026-01-20	warangal
16	1003	Amit Kumar	headphones	5	1500	2026-01-07	warangal
17	1018	Rohit Verma	LAPTOP	2	99999	not-a-date	HYDERABAD

1. Identify missing values in every column

```
df.isnull().sum()
```

```
Order_ID        0
Customer_Name   1
Product         1
Quantity        1
Unit_Price      1
Order_Date      0
Region          1
dtype: int64
```

only columns containing missing values:

```
df.isnull().sum()[df.isnull().sum() > 0]
```

```
Customer_Name    1
Product          1
Quantity         1
Unit_Price       1
Region           1
dtype: int64
```

Total Count :

```
df.isnull().sum().sum()
```

```
np.int64(5)
```

Possible Errors and Solutions

- **Problem:** Missing values can lead to incorrect revenue calculations, incomplete reports, and unreliable business decisions.

- **Solution**: Use df.isnull().sum() to identify missing values in each column. Fill missing values with appropriate values where possible, or remove incomplete records when justified. For important fields such as quantity and unit price, investigate the original records before making corrections to avoid inaccurate revenue calculations.

2. Identify duplicate orders and repeated Order IDs

```
[8]: df[df.duplicated(subset="Order_ID")]
```

[8]:

	Order_ID	Customer_Name	Product	Quantity	Unit_Price	Order_Date	Region
16	1003	Amit Kumar	headphones	5	1500	2026-01-07	warangal

```
[9]: df[df.duplicated(subset="Order_ID", keep=False)]
```

[9]:

	Order_ID	Customer_Name	Product	Quantity	Unit_Price	Order_Date	Region
2	1003	Amit Kumar	headphones	5	1500	2026-01-07	warangal
16	1003	Amit Kumar	headphones	5	1500	2026-01-07	warangal

Removing The Duplicate Records :

```
[11]: df = df.drop_duplicates(subset="Order_ID", keep="first")
```

```
[13]: df[df.duplicated(subset="Order_ID")]
```

[13]:

Order_ID	Customer_Name	Product	Quantity	Unit_Price	Order_Date	Region

```
[ ]:
```

Possible Errors and Solutions

- **Problem:** Duplicate orders can inflate total sales, revenue, and order counts, leading to incorrect business reports and decisions.

- **Solution:** Use df.duplicated(subset="Order_ID", keep=False) to identify all rows with repeated order IDs. Verify the records to determine whether they are genuine duplicates before removing them with drop_duplicates(). If repeated IDs contain different order details, investigate further instead of deleting them blindly.

3. Find the highest-priced product

```
[14]: df["Unit_Price"] = pd.to_numeric(
          df["Unit_Price"], errors="coerce"
      )

[15]: highest_index = df["Unit_Price"].idxmax()

[16]: df.loc[highest_index, ["Product", "Unit_Price"]]

[16]: Product          mobile
      Unit_Price    1558000.0
      Name: 15, dtype: object

[17]: df["Unit_Price"].max()

[17]: np.float64(1558000.0)

[18]: df[df["Unit_Price"]==df["Unit_Price"].max()]
```

[18]:

	Order_ID	Customer_Name	Product	Quantity	Unit_Price	Order_Date	Region
15	1016	divya sharma	mobile	5	1558000.0	2026-01-20	warangal

Possible Errors and Solutions

- **Problem**: Invalid values in the unit price column can cause errors during numerical analysis. Unusually high prices can also produce misleading results and affect revenue calculations.

- **Solution:** Use pd.to_numeric(errors="coerce") to convert invalid text values into missing values. Then use idxmax() to identify the row containing the highest unit price and max() to retrieve the highest price itself. Verify unusually high prices against the original records before correcting them.

-idxmax() returns the index of the row with the highest value, whereas max() returns the highest value itself.

4. Group revenue by city

First, clean the city names and calculate revenue.

```
df["Region"] = df["Region"].str.strip().str.title()

df["Quantity"] = pd.to_numeric(
    df["Quantity"], errors="coerce"
)

df["Revenue"] = df["Quantity"] * df["Unit_Price"]

city_revenue = df.groupby("Region")["Revenue"].sum()

print(city_revenue)
```

```
Region
Hyderabad     488998.0
Khammam       439000.0
Warangal     7819500.0
Name: Revenue, dtype: float64
```

Possible Errors and Solutions

- **Problem:** Revenue may be calculated incorrectly, and sales from the same city may appear under separate names. This can lead to inaccurate city-wise comparisons and business decisions.

- **Solution:** Use str.strip().str.title() to standardize city names and pd.to_numeric(errors="coerce") to convert invalid numerical values into missing values. Calculate revenue by multiplying quantity by unit price, then use groupby() and sum() to calculate city-wise revenue. Check records with missing revenue before interpreting the results, because Pandas sum() skips missing values by default.

Example: " hyderabad ", "HYDERABAD" and "hyderabad" will all become "Hyderabad", ensuring that their revenue is grouped together.